

```
61 GROUP BY Order_ID, Product_ID;
62
63
64 -- 3. Subquery to find items with above-average sales
65 • SELECT order_id, product_id, sales
66 FROM orders
67 WHERE sales > (SELECT AVG(sales) FROM orders)
68 ORDER BY sales DESC;
69
70
71
```

Result Grid		
Filter Rows:	Edits:	Export/Import:
Wrap Cell Content:	Fetch rows:	
order_id	product_id	sales
CA-2014-145317	TEC-MA-10002412	22638.48
CA-2016-118689	TEC-CO-10004722	17499.95
CA-2017-140151	TEC-CO-10004722	13999.96
CA-2017-127180	TEC-CO-10004722	11199.97
CA-2017-166709	TEC-CO-10004722	10499.97
CA-2016-117121	OFF-BI-10000545	9892.74
CA-2014-116904	OFF-BI-10001120	9449.95

```
70 -- 4. To find the highest order for each customer
71 • WITH OrderTotals AS (
72     -- Group items to get true order totals
73     SELECT customer_id, order_id, SUM(sales) AS total_order_sales
74     FROM orders
75     GROUP BY customer_id, order_id
76 )
77 SELECT
78     c.customer_name,
79     ot1.order_id,
80     ROUND(ot1.total_order_sales, 2) AS highest_order_value
```

Result Grid		
Filter Rows:	Export:	Wrap Cell Content:
customer_name	order_id	highest_order_value
Sean Miller	CA-2014-145317	23661.24
Tamara Chand	CA-2016-118689	18302.37
Raymond Buch	CA-2017-140151	14052.48
Tom Ashbrook	CA-2017-127180	13716.46
Becky Martin	CA-2014-139892	10539.90
Hunter Lopez	CA-2017-166709	10499.97
Sanjit Chand	CA-2014-116904	9900.19
Adrian Barton	CA-2016-117121	9892.74

```
91 -- 4. Use CTEs to compute aggregations (total sales per customer)
92 WITH CustomerSales AS (
93     SELECT customer_id, SUM(sales) AS total_customer_sales
94     FROM orders
95     GROUP BY customer_id
96 )
97 SELECT c.customer_name, cs.total_customer_sales
98 FROM CustomerSales cs
99 JOIN customers c ON cs.customer_id = c.customer_id
100 ORDER BY cs.total_customer_sales DESC;
101
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_name	total_customer_sales
▶	Sean Miller	25043.07
	Tamara Chand	19017.85
	Raymond Buch	15117.35
	Tom Ashbrook	14595.62
	Adrian Barton	14355.61
	Sanjit Chand	14142.34
	Ken Lonsdale	14071.92

Result Grid

Form Editor

```
101
102 -- 5. Apply window functions (ROW_NUMBER, RANK)
103 SELECT
104     order_id,
105     product_id,
106     sales,
107     ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY sales DESC) AS row_num_rank,
108     RANK() OVER (PARTITION BY order_id ORDER BY sales DESC) AS gap_rank
109 FROM orders
110 WHERE order_id IN ('CA-2016-152156', 'CA-2016-138688')
111 ORDER BY order_id, sales DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	order_id	product_id	sales	row_num_rank	gap_rank
▶	CA-2016-138688	OFF-LA-10000240	14.62	1	1
	CA-2016-152156	FUR-CH-10000454	731.94	1	1
	CA-2016-152156	FUR-BO-10001798	261.96	2	2

Result Grid

Form Editor

```
113
114 -- 6. Combine JOIN + CTE + Window Functions for final result (customer, total sales, rank)
115 WITH CustomerRevenueCTE AS (
116     -- Step 1: Use a CTE to aggregate all gross sales per unique customer identifier
117     SELECT
118         customer_id,
119         SUM(sales) AS total_sales,
120         COUNT(DISTINCT order_id) AS transaction_count
121     FROM orders
122     GROUP BY customer_id
123 )
```

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
customer_name	segment	aggregate_revenue	transaction_count	customer_rank	
Sean Miller	Home Office	25043.07	5	1	
Tamara Chand	Corporate	19017.85	5	2	
Raymond Buch	Consumer	15117.35	6	3	
Tom Ashbrook	Home Office	14595.62	4	4	
Adrian Barton	Consumer	14355.61	9	5	
Sanjit Chand	Consumer	14142.34	9	6	
Ken Lonsdale	Consumer	14071.92	10	7	
Ken Lonsdale	Consumer	13872.30	6	8	

```
137 -- 7. Solve business queries (top customers, low customers, single-order customers, above-average sales).
138 -- 7.1 Top Customers
139 WITH CustomerSpend AS (
140     SELECT
141         customer_id,
142         SUM(sales) AS total_spent,
143         COUNT(DISTINCT order_id) AS total_orders
144     FROM orders
145     GROUP BY customer_id
146 )
147 SELECT
```

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
customer_name	segment	lifetime_value	total_orders	vip_rank	
Sean Miller	Home Office	25043.07	5	1	
Tamara Chand	Corporate	19017.85	5	2	
Raymond Buch	Consumer	15117.35	6	3	
Tom Ashbrook	Home Office	14595.62	4	4	
Adrian Barton	Consumer	14355.61	9	5	
Sanjit Chand	Consumer	14142.34	9	6	
Ken Lonsdale	Consumer	14071.92	10	7	
Ken Lonsdale	Consumer	13872.30	6	8	

```

155 ORDER BY vip_rank ASC
156 LIMIT 10;
157
158 -- 7.2 Low Customers
159 WITH CustomerSpend AS (
160     SELECT
161         customer_id,
162         SUM(sales) AS total_spent
163     FROM orders
164     GROUP BY customer_id
165 )

```

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
customer_name	segment	total_spent	bottom_rank	
Thais Sissman	Consumer	4.84	1	
Lela Donovan	Corporate	5.30	2	
Mitch Gastineau	Corporate	12.32	3	
Carl Jackson	Corporate	16.52	4	
Roy Skaria	Home Office	22.33	5	
Susan Gilcrest	Corporate	41.08	6	
Ricardo Emerson	Consumer	48.36	7	

```

188 -- 7.4 Above-Average Sales Transactions
189 SELECT
190     o.order_id,
191     c.customer_name,
192     p.product_name,
193     ROUND(o.sales, 2) AS transaction_sales
194 FROM orders o
195 JOIN customers c ON o.customer_id = c.customer_id
196 JOIN products p ON o.product_id = p.product_id
197 WHERE o.sales > (SELECT AVG(sales) FROM orders)
198 ORDER BY o.sales DESC;

```

Result Grid				
Filter Rows:				
Export:				
Wrap Cell Content:				
Fetch rows:				
order_id	customer_name	product_name	transaction_sales	
CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferen...	22638.48	
CA-2016-118689	Tamara Chand	Canon imageCLASS 2200 Advanced Copier	17499.95	
CA-2017-140151	Raymond Buch	Canon imageCLASS 2200 Advanced Copier	13999.96	
CA-2017-127180	Tom Ashbrook	Canon imageCLASS 2200 Advanced Copier	11199.97	
CA-2017-166709	Hunter Lopez	Canon imageCLASS 2200 Advanced Copier	10499.97	
CA-2016-117121	Adrian Barton	GBC Ibimaster 500 Manual ProClick Binding System	9892.74	
CA-2014-116904	Sanjit Chand	Ibico EPK-21 Electric Binding System	9449.95	
US-2016-107440	Bill Shonely	3D Systems Cube Printer, 2nd Generation, Man	8000.03	